NEX

MACHINE LEARNING

Knowledge Base for a Chat-Based Study Assistant

Purpose
Provide NEX with clear, reliable foundational information to answer student questions about machine learning, common algorithms, model evaluation, and beginner Python practice.

How NEX should use this document

- Answer in simple language first, then add technical detail when the user asks for it.
- Explain unfamiliar terms and use small examples or tables where useful.
- For code questions, state required libraries, expected input format, and how to run the code.
- Do not invent results. If performance depends on a dataset or experiment, explain that it must be measured.
- Mention limitations, assumptions, and responsible-use considerations when relevant.

Audience: students and beginners studying AI/ML. **Scope:** core concepts, supervised and unsupervised learning, algorithms, data preparation, evaluation, practical Python, FAQs, and learning resources.

Version 1.0 • Prepared as a reusable knowledge-base document

1. Introduction to Machine Learning

Machine Learning (ML) is a field of artificial intelligence that enables computers to learn patterns from data and use those patterns to make predictions, classifications, or decisions. Instead of writing a separate rule for every situation, developers train a model using examples.

Artificial intelligence (AI) is the broader field of creating systems that perform tasks associated with human intelligence. Machine learning is one approach within AI; deep learning is a subset of ML that uses neural networks with multiple layers.

Key terms

Term	Meaning	Example
Dataset	A collection of related examples or records.	A table of house sales.
Feature	An input variable used by a model.	House area, location, bedrooms.
Label / target	The value a model is trained to predict in supervised learning.	Sale price or spam/not spam.
Model	A learned mathematical or computational pattern.	A decision tree predicting loan default.
Training	The process of learning model parameters from data.	Fitting a classifier to labelled examples.
Inference / prediction	Using a trained model on new input.	Predicting a new email's class.
Generalization	How well a model performs on unseen data.	Correctly classifying new emails.

Typical machine-learning workflow

- Define the problem and the outcome to predict.
- Collect relevant, lawful, representative data.
- Explore and clean the data; handle missing values and errors.
- Split data into training, validation (when needed), and test sets.
- Prepare features and train a suitable model.
- Evaluate performance with metrics that match the problem.
- Tune and compare models without using the test set repeatedly.
- Deploy, monitor, and update the model as data or needs change.

Important: A model can perform well on training data and still fail on new data. The goal is reliable generalization, not simply memorizing examples.

2. Main Types of Machine Learning

The learning type is chosen based on what information is available and what the task needs to achieve.

Type	Training information	Common tasks	Examples
Supervised learning	Examples include input features and known target labels.	Classification and regression.	Predict whether a message is spam; estimate a home price.
Unsupervised learning	Inputs are available without target labels.	Clustering, dimensionality reduction, anomaly exploration.	Group customers by purchasing patterns.
Semi-supervised learning	A small labelled set plus a larger unlabelled set.	Classification when labelling is expensive.	Image classification with only some images labelled.
Reinforcement learning	An agent interacts with an environment and receives rewards or penalties.	Sequential decisions and control.	A simulated robot learning to navigate.

Classification vs regression

Task	Output	Example algorithms	Typical metrics
Classification	A category or class, sometimes with class probabilities.	Logistic Regression, Decision Tree, SVM, k-NN, Random Forest.	Accuracy, precision, recall, F1, ROC-AUC.
Regression	A numeric value.	Linear Regression, Decision Tree Regressor, Random Forest Regressor.	MAE, MSE, RMSE, R^2.

Clustering groups examples by similarity without known labels. Dimensionality reduction represents data with fewer variables while trying to retain useful structure. Anomaly detection identifies examples that differ strongly from typical patterns; unusual examples are not necessarily errors or fraud.

Where ML is used

- Healthcare: supporting image analysis or risk estimation; clinical decisions require appropriate validation and professional oversight.
- Finance: fraud detection, credit-risk estimation, and forecasting, with attention to fairness and regulation.
- Retail: demand forecasting, recommendation systems, and customer segmentation.
- Manufacturing: predictive maintenance and visual quality inspection.
- Cybersecurity: suspicious activity detection and spam or malware classification.
- Education: learning-resource recommendations and analysis of learning patterns.

Use a simple baseline first. More complex models are not automatically better; compare them using appropriate metrics, data quality, interpretability, cost, and deployment needs.

3. Common Machine-Learning Algorithms

Algorithm	Main idea	Strengths	Limitations / notes
Linear Regression	Fits a linear relationship to predict a numeric target.	Simple, fast, interpretable baseline.	May underfit nonlinear patterns; sensitive to influential outliers.
Logistic Regression	Models class probability using a logistic function.	Efficient, interpretable baseline; probability output.	Linear decision boundary unless features are transformed.
Decision Tree	Splits data into rule-based branches.	Easy to explain; handles nonlinear patterns and mixed feature types.	Can overfit; small data changes can change the tree.
Random Forest	Combines predictions from many decision trees.	Often strong general-purpose performance; reduces single-tree variance.	Less transparent than one tree; larger model.
Support Vector Machine (SVM)	Finds a decision boundary with a large margin; kernels can model nonlinear boundaries.	Effective in many high-dimensional settings.	Scaling is important; kernel SVMs may be costly on large datasets.
k-Nearest Neighbors (k-NN)	Predicts from the labels/values of nearby training examples.	Intuitive and simple; little explicit training.	Prediction can be slow; sensitive to scaling and choice of k.
K-Means	Assigns points to a chosen number of clusters around centroids.	Simple and efficient for compact clusters.	Must choose k; sensitive to scaling and outliers; assumes centroid-based groups.
Hierarchical Clustering	Builds a hierarchy of nested clusters.	Dendrogram can help explore cluster structure.	Can be expensive on large datasets; linkage choice matters.
Neural Network / MLP	Learns layered nonlinear transformations.	Can model complex relationships with enough data and tuning.	Needs careful tuning; less interpretable; may need more computation.

How to choose an algorithm

- Start with the task: classification, regression, clustering, or another goal.
- Consider data size, feature types, missing values, and whether scaling is needed.
- Use interpretability requirements to guide the choice.
- Compare a baseline and one or more alternatives using the same split and metrics.
- Consider inference speed, memory, maintenance, and the cost of mistakes.

Rule of thumb: Decision Trees are useful for explainable rules; Logistic Regression is a strong classification baseline; Random Forests often improve robustness; SVMs can work well on scaled datasets; K-Means is a starting point for exploratory clustering. These are starting points, not guarantees.

4. Data Preparation and Model Training

Data quality strongly affects model quality. Before training, inspect the dataset and understand what each column represents, how it was collected, and whether it is appropriate for the intended use.

Common preparation steps

- Missing values: remove rows only when justified, or impute using a suitable statistic or domain-specific method. Fit imputation rules on training data only.
- Duplicates: identify repeated records and decide whether they are true duplicates or meaningful repeated events.
- Outliers: investigate whether values are errors or legitimate rare cases before removing or changing them.
- Categorical variables: use one-hot encoding or another suitable encoding for non-numeric categories.
- Scaling: StandardScaler or MinMaxScaler can help algorithms such as SVM, k-NN, Logistic Regression, and neural networks. Tree-based models generally do not require feature scaling.
- Feature selection/engineering: create or select meaningful inputs without using information that would be unavailable at prediction time.
- Class imbalance: inspect class counts. Consider class weights, resampling, threshold adjustment, and metrics beyond accuracy when appropriate.

Train, validation, and test data

Split	Purpose	Good practice
Training set	Fits model parameters.	Use only this data to fit preprocessing and model parameters.
Validation set	Helps choose settings and compare candidate models.	Use for hyperparameter tuning and model selection. Cross-validation can be used when data is limited.
Test set	Estimates final performance on unseen data.	Keep separate until final evaluation; do not repeatedly tune against it.

Data leakage

Data leakage happens when information that would not legitimately be available at prediction time influences training or evaluation. Examples include scaling the full dataset before splitting, including a future outcome as a feature, or accidentally allowing records from the same person to appear in both train and test sets. Leakage can produce unrealistically high scores.

Reproducibility checklist

- Set a random seed where appropriate (for example, random_state=42).
- Record the dataset source, target column, feature columns, split method, and metrics.
- Fit transformations on training data and apply the fitted transformations to validation/test data.
- For time-dependent data, split chronologically when that reflects real-world prediction.
- For grouped data, keep related records in the same split when needed.

5. Evaluating Machine-Learning Models

Evaluation must match the task and the cost of different errors. A single metric rarely tells the whole story.

Classification metrics

Metric	Meaning	When useful / caution
Accuracy	Correct predictions divided by all predictions.	Easy to understand, but can mislead when classes are imbalanced.
Precision	Of predicted positives, the fraction that are truly positive.	Important when false positives are costly.
Recall (sensitivity)	Of actual positives, the fraction correctly identified.	Important when missing a positive case is costly.
F1-score	Harmonic mean of precision and recall.	Useful when balancing precision and recall; does not include true negatives directly.
Confusion matrix	Counts true positives, true negatives, false positives, and false negatives.	Shows the kinds of errors made by a classifier.
ROC-AUC	Measures ranking discrimination across thresholds.	Useful for comparing scores across thresholds; interpret carefully on highly imbalanced data.

Confusion matrix (binary classification)

	Predicted positive	Predicted negative
Actual positive	True Positive (TP)	False Negative (FN)
Actual negative	False Positive (FP)	True Negative (TN)

Common formulas

Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 = 2 × (Precision × Recall) / (Precision + Recall)

Regression metrics

Metric	Meaning	Interpretation
MAE	Mean absolute error.	Average absolute difference between predictions and true values, in target units.
MSE	Mean squared error.	Squares errors, penalizing large errors more strongly.
RMSE	Square root of MSE.	Error is expressed in the target's units.
R^2	Coefficient of determination.	Measures fit relative to predicting the target mean; can be negative on test data.

Always report the evaluation setup with the metric: dataset/split, target, class balance if relevant, and whether results are from validation or test data. Do not compare scores from incompatible splits as if they were directly equivalent.

6. Overfitting, Underfitting, and Python Practice

Concept	What it means	Possible response
Overfitting	Model learns training-specific noise or details; training performance is much better than unseen-data performance.	Use cross-validation, simplify the model, regularize, collect more data, or use early stopping where appropriate.
Underfitting	Model is too simple or insufficiently trained to capture important patterns.	Use more informative features, a more flexible model, or suitable training/tuning.
Bias–variance trade-off	A model's errors reflect a balance between overly simple assumptions and sensitivity to the training sample.	Compare generalization performance, not training score alone.

Beginner example: Decision Tree on the Iris dataset

This example loads the built-in Iris dataset, splits it into training and test sets, trains a Decision Tree classifier, and prints accuracy and a classification report. It does not require downloading a CSV file.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load sample dataset
iris = load_iris()
X = iris.data       # four numeric flower measurements
y = iris.target     # three flower classes

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = DecisionTreeClassifier(random_state=42, max_depth=3)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred,
                            target_names=iris.target_names))
```

How to run

1. Install Python and scikit-learn if they are not already installed: `pip install scikit-learn`
2. Save the code as `iris_tree.py`.
3. Run it in a terminal: `python iris_tree.py`.
4. Your printed score may vary if you change the split, model settings, or dataset. Accuracy alone is not a complete evaluation.

Useful Python libraries

Library	Main use
NumPy	Numerical arrays and mathematical operations.
pandas	Loading, cleaning, and exploring tabular data.
Matplotlib / Seaborn	Charts and exploratory data visualisation.
scikit-learn	Preprocessing, classical ML models, splitting, metrics, and model selection.
TensorFlow / Keras, PyTorch	Building and training neural networks and deep-learning models.

7. Glossary, FAQs, and Learning Resources

Quick glossary

Term	Short definition
Hyperparameter	A setting chosen before or during model selection, such as tree depth or learning rate.
Cross-validation	Repeatedly trains and validates across different data folds to estimate model performance.
Regularization	A technique that discourages overly complex models.
Epoch	One pass through the training data in many neural-network training setups.
Batch size	Number of training examples processed before a parameter update in batch-based training.
Learning rate	Controls the size of parameter updates during optimization.
Standardization	Transforms features using training-set mean and standard deviation.
One-hot encoding	Represents a category using binary indicator columns.
Class imbalance	A situation where some target classes occur much more often than others.
Deployment	Making a trained model available for use in an application or workflow.

Frequently asked questions

Is ML the same as AI?
No. AI is the broader field; ML is one approach within AI. Deep learning is a subset of ML.

Which algorithm is best?
There is no universally best algorithm. The choice depends on the task, data, constraints, interpretability needs, and measured performance.

Why scale features?
Distance- and gradient-based algorithms can be affected by feature magnitudes. Scaling prevents large-unit features from dominating for reasons unrelated to their usefulness. Tree-based methods usually do not need scaling.

Can high accuracy be misleading?
Yes. On imbalanced data, a model can predict the majority class and still achieve high accuracy. Inspect precision, recall, F1, and the confusion matrix as appropriate.

Does a good test score guarantee real-world success?
No. Real-world data may differ from the test data. Monitor performance, data drift, fairness, privacy, and the consequences of errors after deployment.

Learning resources

- scikit-learn user guide: https://scikit-learn.org/stable/user_guide.html
- scikit-learn tutorials: https://scikit-learn.org/stable/getting_started.html
- Google Machine Learning Crash Course: https://developers.google.com/machine-learning/crash-course
- pandas documentation: https://pandas.pydata.org/docs/
- NumPy documentation: https://numpy.org/doc/

Responsible ML reminders

- Respect privacy and data-use permissions; avoid exposing sensitive data.
- Check whether data or model outcomes systematically disadvantage groups.
- Explain uncertainty and limitations; do not present predictions as guaranteed facts.
- For high-impact areas such as healthcare, finance, or employment, require appropriate expert review and governance.

End of knowledge base. NEX should use this document as a foundation, adapt explanations to the user's level, and recommend checking current official documentation when software APIs or best practices may have changed.